

Guía Integral de Estudio: Requerimientos Ágiles, User Stories y Estimaciones

Ingeniería y Calidad de Software (ISW) — Universidad Tecnológica Nacional (UTN FRC)

Fuentes Consolidadas: Filminas de Cátedra (03 Rqs Ágiles USER STORIES & 04 Estimaciones Ágiles), Anotaciones de Clase, Resumen Histórico de Alumnos y Guías de TP

Requerimientos Ágiles

Fuentes de Cátedra: Filmina 03 (Diaps. 2–18) | Resumen Teórico Final (Págs. 36–38) | Apuntes de Clase

En la ingeniería de software clásica se consideraba que la mayor dificultad radicaba en las complejidades tecnológicas de la programación. Sin embargo, la historia de la disciplina —marcada por la histórica *Crisis del Software*— demostró que **el factor crítico de fracaso en los proyectos radica en no satisfacer las expectativas del cliente**: construir un software técnicamente impecable pero que no es lo que el cliente realmente necesitaba ni resuelve su problema de negocio.

Frente a este escenario, los enfoques ágiles y Lean transforman de raíz la gestión de requerimientos, sustituyendo la especificación contractual y estática por un proceso continuo de descubrimiento, conversación y entrega temprana de valor.

Pregunta Clave de Examen Parcial: ¿Por qué son ágiles los requerimientos ágiles?

Los requerimientos ágiles reciben la denominación de **ágiles** porque **no se congelan ni se definen de forma exhaustiva al inicio del proyecto, sino que son adaptativos y emergentes**:

1. **Son emergentes**: Se reconoce que más del 50% de las necesidades reales solo se descubren cuando el usuario interactúa con incrementos reales de software funcionando.
2. **Se gestionan Just-in-Time**: Se analizan y refinan en el *último momento responsable*, evitando el desperdicio de detallar funcionalidades con meses de anticipación que cambiarán antes de implementarse.
3. **Foco en el Outcome (Valor) sobre el Output (Volumen)**: El éxito no se mide por la cantidad de líneas de código o pantallas producidas, sino por el beneficio e impacto concreto que generan en el negocio.
4. **Gobernados por la Conversación continua**: Las User Stories no son contratos cerrados, sino tokens para propiciar el diálogo directo cara a cara entre el equipo y el Product Owner.
5. **Aceptan el cambio como ventaja competitiva**: Los cambios no se gestionan burocráticamente como errores o anomalías del plan, sino como oportunidades de aprendizaje para perfeccionar el producto.

¿Qué es Ágil?

Fuentes de Cátedra: Filmina 03 (Diaps. 2–5) | Resumen Teórico Final (Pág. 36) | Manifiesto Ágil

Ágil NO es una metodología ni un proceso. Las metodologías tradicionales suelen ser prescriptivas, rígidas y orientadas a detallar el *cómo* paso a paso. **Ágil es una filosofía y un marco de pensamiento (mindset)** fundamentado en un conjunto explícito de valores y principios que guían el desarrollo de productos en entornos inciertos y complejos.

DISCIPLINAS Y VALORES DE LOS EQUIPOS ÁGILES:

- **Planificación continua y multi-nivel**: La planificación no es un evento único al inicio; ocurre de manera continua a nivel estratégico, de release, de sprint y en la sincronización diaria (Daily).

- **Equipos facultados, auto-organizados y multidisciplinarios:** Cuentan con autonomía formal para decidir el *cómo* construir la solución y poseen todas las destrezas necesarias para entregar valor de punta a punta sin dependencias externas.
- **Entregas frecuentes, iterativas y priorizadas:** Se generan incrementos terminados de software cada 2 a 4 semanas, enfocados en satisfacer primero los requerimientos de mayor valor de negocio.
- **Prácticas de ingeniería disciplinadas:** La agilidad no es indisciplina ni improvisación. Exige un rigor técnico supremo mediante TDD (Test-Driven Development), refactorización continua y programación en pares.
- **Integración continua (CI):** El código se integra y verifica automáticamente de forma periódica, garantizando la detección de incompatibilidades en horas y no al final del proyecto.
- **Testing concurrente:** Las pruebas de calidad no constituyen una etapa posterior al desarrollo; se diseñan y ejecutan de forma paralela y simultánea a la programación.

El balance ágil: La agilidad busca el equilibrio óptimo entre la ausencia total de proceso (el caos y la anarquía) y el exceso de proceso (la burocracia paralizante):

- Los métodos ágiles son **adaptables** en lugar de ser **predictivos**.
- Los métodos ágiles están **orientados a las personas** en lugar de estar **orientados a los procesos**.

Manifiesto Ágil

Fuentes de Cátedra: Filmina 03 (Diaps. 6–7) | Resumen Teórico Final (Pág. 36) | Agile Alliance (2001)

En febrero de 2001, en Snowbird (Utah), 17 destacados líderes y practicantes del desarrollo de software redactaron el **Manifiesto para el Desarrollo Ágil de Software**. Su propósito fue acordar un marco común sobre el **qué** y el **por qué** del desarrollo, concediendo total libertad respecto al **cómo** (las prácticas singulares de Scrum, XP, Crystal, etc.).

El Manifiesto Ágil postula formalmente que el desarrollo de software pertenece al dominio de los **procesos empíricos**, en contraposición directa a los **procesos definidos**:

Criterio	Control de Procesos Definidos (Tradicional)	Control de Procesos Empíricos (Ágil / Lean)
Premisa de partida	Asume que el problema es completamente predecible y comprendido de antemano. Dado un conjunto de entradas, el proceso siempre produce la misma salida esperada.	Asume que el desarrollo de software es un proceso intelectual, creativo y complejo, intrínsecamente incierto e imposible de predecir de antemano.
Figura de autoridad	Centrada en un jefe de proyecto o supervisor con autoridad formal para comandar, asignar tareas y controlar el cumplimiento rígido de las etapas.	Equipos autogestionados y empoderados . No existe un jefe de proyecto; las decisiones sobre cómo construir la solución son colectivas.
Pilares de gestión	Planificación detallada inicial, seguimiento estricto de la línea base (Baseline) y burocracia en el Control de Cambios.	Transparencia, Inspección y Adaptación. El conocimiento genuino proviene de la experiencia empírica y la observación de hechos reales.
Rol de las personas	Visión fabril/taylorista: las personas se consideran «recursos intercambiables». Se asume que el proceso garantiza el resultado independientemente de quién lo ejecute.	Las personas y sus interacciones son el elemento primordial. Se reconoce que la productividad y la calidad dependen críticamente de las dinámicas humanas.

Filosofía Ágil

Fuentes de Cátedra: Filmina 03 (Diaps. 8–9) | Resumen Teórico Final (Págs. 36–37) | Apuntes de Clase

La filosofía ágil sostiene que, en entornos turbulentos, la única forma racional de gestionar el riesgo es la **entrega temprana y continua de valor** mediante un **ciclo de vida iterativo-incremental**:

- El software no se entrega como un bloque monolítico al término de meses o años, sino en pequeños incrementos funcionales desplegables cada 2 o 3 semanas.
- El cliente y los usuarios participan activamente a lo largo de todo el ciclo, proporcionando una retroalimentación continua que nutre los puntos de inspección y adaptación.
- **Diferencia clave con RUP / PUD:** Aunque el Proceso Unificado (RUP) utiliza un ciclo de vida formalmente iterativo e incremental, históricamente fue concebido como un **proceso definido** gobernado por fases secuenciales fijas (Incepción, Elaboración, Construcción, Transición) y compuertas de aprobación formal (*phase gates*) que buscan predecir el proyecto. En contraposición, los marcos ágiles adoptan el control empírico de procesos en su totalidad.

Valores Ágiles

Fuentes de Cátedra: Filmina 03 (Diaps. 7–8) | Resumen Teórico Final (Pág. 36) | Manifiesto Ágil

El Manifiesto Ágil consagra cuatro postulados fundamentales estructurados bajo la regla de valoración:

«*Valoramos más los elementos de la izquierda (arriba) que los elementos de la derecha (abajo)*».



Infografía oficial: Los 4 valores del Manifiesto Ágil

DESGLOSE Y JUSTIFICACIÓN DE LOS 4 VALORES:

1. Individuos e interacciones sobre procesos y herramientas:

- Las herramientas más avanzadas y los procesos más rigurosos son totalmente inútiles si la comunicación entre las personas falla.
- El éxito de un proyecto de software radica en la motivación del equipo, la confianza mutua y la capacidad de autogestión para resolver problemas en conjunto.

2. Software funcionando sobre documentación exhaustiva:

- La documentación técnica voluminosa (documentos SRS de cientos de páginas) no genera por sí misma ningún retorno de inversión ni demuestra avance real.
- Lo único que genera valor genuino para el cliente y valida empíricamente el progreso es el software operativo, libre de defectos y ejecutable.
- *Aclaración:* Esto no significa «cero documentación», sino generar únicamente documentación ágil, suficiente y valiosa (*Just Enough*).

3. Colaboración con el cliente sobre negociación contractual:

- En el enfoque tradicional, el cliente y el proveedor se sitúan en veredas opuestas, blindados detrás de especificaciones cerradas y penalidades legales.

- En Agile, el cliente se incorpora como socio directo en el día a día a través del rol del **Product Owner (PO)**, co-creando la solución y priorizando juntos el valor.

4. Respuesta ante el cambio sobre seguir un plan:

- En un entorno cambiante, aferrarse rígidamente a un plan obsoleto garantiza el fracaso del producto.
- Se asume desde el inicio que los requerimientos van a evolucionar a medida que el mercado cambie y el usuario interactúe con el sistema. La adaptabilidad rápida es la clave de la supervivencia competitiva.

⚠ Aclaración Fundamental de Cátedra sobre los Valores

El propio Manifiesto concluye advirtiendo: «Aunque reconocemos valor en los elementos de la derecha, valoramos MÁS los de la izquierda».

Esto significa que los procesos, las herramientas, la documentación, los contratos y los planes siguen existiendo, pero se subordinan estrictamente al servicio de las personas, del software funcionando y de la entrega continua de valor.

Los 12 Principios del Manifiesto Ágil

Fuentes de Cátedra: Filmina 03 (Diaps. 10–12) | Resumen Teórico Final (Págs. 37–38) | Agile Manifesto

Los cuatro valores fundamentales se operacionalizan a través de los 12 principios rectores del Manifiesto Ágil:

1. **Nuestra mayor prioridad es satisfacer al cliente:** Mediante la entrega temprana y continua de software con valor. Por cada release del producto, el cliente debe percibir un retorno tangible. *(Vinculado a los Valores 3 y 4).*
2. **Aceptamos que los requerimientos cambien:** Incluso en etapas avanzadas del desarrollo. Los procesos ágiles aprovechan el cambio como una ventaja competitiva para el cliente, desafiando la utopía de congelar requerimientos. *(Vinculado a los Valores 3 y 4).*
3. **Entregamos software funcional frecuentemente:** Con una periodicidad de entre dos semanas y dos meses, mostrando una marcada preferencia por la escala de tiempo más corta posible. *(Vinculado al Valor 2).*
4. **Los responsables de negocio y los desarrolladores deben trabajar juntos diariamente:** Durante todo el ciclo del proyecto. El PO aporta la visión comercial y el presupuesto (*accountability de negocio*), mientras que el equipo decide la arquitectura técnica (*accountability técnica*). No hay intermediarios burocráticos. *(Vinculado a los Valores 1 y 3).*
5. **Desarrollamos proyectos en torno a individuos motivados:** Hay que brindarles el entorno, las herramientas y el respaldo que necesitan, confiando plenamente en su compromiso para realizar la tarea. *(Vinculado al Valor 1).*
6. **El método más eficiente y efectivo de comunicar información es la conversación cara a cara:** La interacción presencial (o mediante videollamada rica) transmite información verbal, gestual y contextual inmediata, reduciendo drásticamente las malas interpretaciones. *(Vinculado al Valor 1).*
7. **El software funcionando es la medida primordial de progreso:** No se mide el avance por porcentaje de tareas iniciadas, horas quemadas o documentos escritos, sino por funcionalidades terminadas y operativas. *(Vinculado al Valor 2).*
8. **Los procesos ágiles promueven el desarrollo sostenible:** Los patrocinadores, desarrolladores y usuarios deben ser capaces de mantener un ritmo de trabajo constante de forma indefinida, erradicando el trabajo nocturno descontrolado (*crunch*) y el desgaste humano (*burnout*). *(Vinculado a los Valores 2 y 4).*
9. **La atención continua a la excelencia técnica y al buen diseño mejora la agilidad:** La calidad interna del software (código limpio, arquitectura desacoplada, pruebas automatizadas) no es negociable; es el cimiento que permite seguir cambiando el sistema rápidamente en el futuro. *(Vinculado al Valor 2).*

10. **La simplicidad es esencial:** Definida magistralmente como «*el arte de maximizar la cantidad de trabajo NO realizado*». Implica no programar nada que no sea estrictamente requerido ahora (principio YAGNI: *You Aren't Gonna Need It*). (*Vinculado a los Valores 2 y 3*).
11. **Las mejores arquitecturas, requerimientos y diseños emergen de equipos auto-organizados:** Quienes están más cerca de las líneas de código y de los usuarios son los más calificados para proponer las soluciones más óptimas y elegantes. (*Vinculado al Valor 1*).
12. **A intervalos regulares, el equipo reflexiona sobre cómo ser más efectivo y ajusta su comportamiento:** A través de la ceremonia de Retrospectiva, el equipo inspecciona su dinámica de trabajo y adopta mejoras concretas para optimizar su desempeño continuo. (*Vinculado a los Valores 3 y 4*).

Requerimientos en Agile: Validación sobre Verificación y Output vs. Outcome

Fuentes de Cátedra: Filmina 03 (Diaps. 13–18) | Resumen Teórico Final (Pág. 38) | Apuntes de Clase (Requerimientos en Ágil)

Fuentes de Cátedra: Filmina 03 (Diaps. 13–18) | Resumen Teórico Final (Pág. 38) | Apuntes de Clase (Requerimientos en Ágil)

En el paradigma ágil se produce una auténtica transformación conceptual respecto a cómo se conciben los requerimientos:

1. VALIDACIÓN (CONSTRUIR EL PRODUCTO CORRECTO) SOBRE VERIFICACIÓN (CONSTRUIRLO CORRECTAMENTE):

Tradicionalmente se ponía el foco en la **verificación**: que el código no arroje excepciones, compile limpiamente, tenga cobertura y respete la arquitectura. Sin embargo, en Agile entendemos que **entregar código sin errores técnicos no alcanza**: si el sistema hace a la perfección algo que el cliente no necesita, el proyecto es un fracaso absoluto. Agile prioriza la **validación**: asegurar continuamente que lo construido responde al problema real del negocio.

2. LA DISTINCIÓN FUNDAMENTAL: OUTPUT VS. OUTCOME:

- **Output (Salida / Producción de código):** Es el volumen tangible de artefactos entregados: cantidad de líneas de código, número de pantallas desarrolladas, tickets cerrados o funcionalidades entregadas.
- **Outcome (Resultado / Beneficio de Negocio):** Es el cambio de comportamiento observable en el usuario, la resolución de un dolor operativo o el impacto financiero concreto generado en la organización gracias a ese output.
- **El Mantra Ágil de Requerimientos:** «*Maximizar el Outcome minimizando el Output*». El objetivo no es saturar el sistema con código y pantallas, sino encontrar la forma más simple posible de generar el mayor beneficio para el negocio con el menor esfuerzo de construcción.

3. EL CICLO VIRTUOSO DE REQUERIMIENTOS ÁGILES:

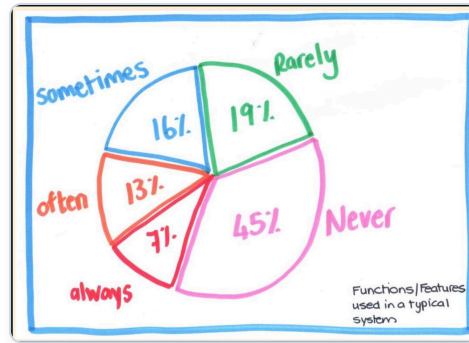
Entender el Valor de Negocio → Usar Historias y Modelos Visuales →
Determinar "Sólo lo Suficiente" (Just-in-Time) → Inspeccionar Feedback y Repetir

El Costo del Tradicional BRUF (Big Requirements Up Front)

Fuentes de Cátedra: Filmina 03 (Diaps. 19–21) | Resumen Teórico Final (Págs. 39–40) | Standish Group CHAOS Study

Fuentes de Cátedra: Filmina 03 (Diaps. 19–21) | Resumen Teórico Final (Págs. 39–40) | Standish Group CHAOS Study

El enfoque tradicional insiste en el **BRUF (Big Requirements Up Front)**, obligando al equipo a intentar especificar el 100% de los requerimientos de forma previa al desarrollo.



Frecuencia de uso de funcionalidades en software (Estudio de Jim Johnson - Standish Group, XP 2002)

EL DESPERDICIO SISTÉMICO DEL BRUF:

Los estudios empíricos de la industria (Standish Group / XP 2002) demuestran que los productos considerados «exitosos» bajo desarrollo tradicional albergan una enorme cantidad de desperdicio:

- 45% de las funcionalidades construidas **NUNCA** son utilizadas por los usuarios.
- 19% se utilizan raramente.
- 16% se utilizan algunas veces.
- Solo el 20% se utilizan frecuentemente o siempre.

Conclusión Contundente: En los proyectos gobernados por BRUF, el **64% del software desarrollado es puro desperdicio (waste)**. El ejemplo cotidiano más contundente es Microsoft Excel: la inmensa mayoría de las organizaciones utiliza menos del 10% de su capacidad total; el restante 90% costó millones de dólares en desarrollo y mantenimiento, pero no aporta valor cotidiano.

LA FALACIA DEL 100% Y LOS REQUERIMIENTOS EMERGENTES:

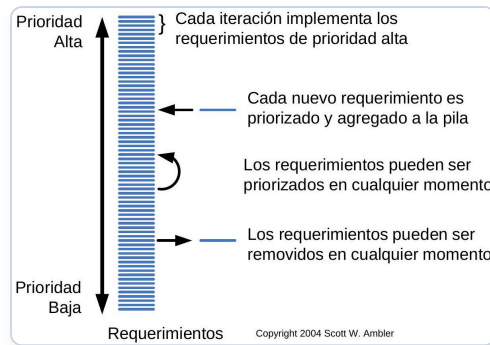
El agilismo demuestra que no solo no es necesario tener el 100% de la especificación inicial, sino que **es fácticamente imposible lograrlo**:

- Al menos el **50% de los requerimientos son emergentes**: aparecen y se descubren únicamente cuando el cliente y los usuarios interactúan de verdad con incrementos reales de software funcionando.
- Respecto al otro 50%, estamos en condiciones de identificarlo al principio **para definir la base inicial (o MVP - Producto Mínimo Viable) con la cual arrancar**, priorizando solo lo que realmente aporta valor esencial y permitiendo que todo lo demás evolucione con base en el feedback real de la experiencia.

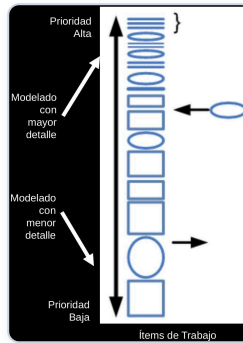
Gestión Ágil de Requerimientos: Backlog DEEP, Requerimientos JIT y Persistencia

Fuentes de Cátedra: Filmina 03 (Diaps. 22–24) | Resumen Teórico Final (Págs. 40–41) | Mike Cohn & Roman Pichler

En los entornos ágiles, los requerimientos cambiantes dejan de ser una amenaza y se convierten en una **ventaja competitiva** si el equipo tiene la disciplina y el marco de trabajo adecuado para actuar sobre ellos.

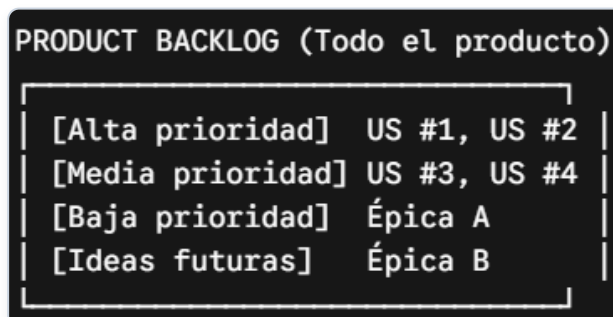


El Product Backlog como un iceberg: Granularidad fina arriba, granularidad gruesa abajo



La pila ordenada de necesidades de negocio

Elemento del Diagrama	Concepto en Agile / Scrum	Qué representa formalmente
Toda la columna vertical (la pila)	Product Backlog	Es una lista ordenada y viva de todas las características, requerimientos, mejoras y correcciones del software. Es el artefacto principal cuya responsabilidad, ordenamiento y mantenimiento recae exclusivamente en el Product Owner .
Cada rayita individual	Product Backlog Item (PBI)	Una necesidad o requerimiento puntual de negocio que habitualmente se redacta bajo la técnica de User Story .
La posición relativa (arriba vs. abajo)	Prioridad y Granularidad	Cuanto más arriba está un ítem, mayor valor de negocio representa y mayor es su nivel de refinamiento y detalle (granularidad fina, listo para implementarse). Cuanto más abajo, menor prioridad tiene y su definición es más amplia y abstracta (Épicas o Temas).



Refinamiento progresivo de requerimientos según prioridad

PRINCIPIOS DE GESTIÓN DEL PRODUCT BACKLOG:

- **Principio Just-in-Time:** No invertimos tiempo ni recursos al inicio del proyecto detallando requerimientos que quizás se implementen dentro de 8 meses. Se analizan a medida que suben en la pila y se preparan justo antes de ingresar al Sprint.

- **Enfoque en los ítems superiores:** El equipo y el Product Owner concentran su energía en los PBIs que están en la cima de la pila, asegurando que estén debidamente estimados y desglosados.
- **Dinamismo absoluto (Gestión del PO):** El Product Owner puede agregar nuevos requerimientos, repriorizar el orden de la pila o eliminar ítems innecesarios en cualquier instante de la vida del proyecto.

4. CONTENEDOR Y PERSISTENCIA DE LOS REQUERIMIENTOS: ERS VS. PRODUCT BACKLOG:

Un concepto teórico modular evaluado en la página 41 del resumen de cátedra:

- **Enfoque Tradicional (Documental):** La Especificación de Requerimientos de Software (ERS) contiene todos los requerimientos a lo largo del ciclo de vida. Cualquier cambio exige renegociar y actualizar la ERS formalmente. Es el artefacto supremo permanente.
- **Enfoque Ágil (Software Funcionando):** El **Product Backlog** es un contenedor puramente TRANSITORIO de requerimientos. A medida que las historias se desarrollan y se completan:
- Persistencia Real en el Producto:
 - El requerimiento en sí **persiste en el software funcionando** (código compilado y desplegado).
 - El detalle fino de valor y las condiciones de satisfacción **persisten en los casos de prueba de aceptación automatizados** construidos por el equipo.

5. ATRIBUTOS ESENCIALES DEL PRODUCT BACKLOG (CRITERIOS DEEP DE ROMAN PICHLER Y MIKE COHN):

- **Detailed Appropriately (Detallado Apropiadamente):** Las historias de usuario en la cima del Backlog (las que entrarán en el próximo Sprint) están desglosadas con gran detalle, criterios de aceptación finos y cumplen con el DoR. Las que están más abajo son grandes épicas difusas que se refinarán en el futuro.
- **Estimated (Estimado):** Todos los ítems que serán planificados cuentan con una estimación de tamaño relativo en Story Points consensuada por el equipo.
- **Emergent (Emergente y Dinámico):** El Backlog nunca está cerrado ni completo; es un artefacto vivo que incorpora nuevos descubrimientos, suprime ítems descartados y muta con base en el feedback real de los incrementos.
- **Prioritized (Priorizado Estrictamente):** Ordenado secuencialmente de arriba hacia abajo según el valor aportado al negocio, la urgencia de mercado, la reducción de riesgos y las dependencias técnicas.

Tipos y Niveles de Requerimientos: Dominio del Problema vs. Dominio de la Solución

Fuentes de Cátedra: Filmina 03 (Diaps. 25–28) | Resumen Teórico Final (Pág. 41) | Apuntes de Clase

Fuentes de Cátedra: Filmina 03 (Diaps. 25–28) | Resumen Teórico Final (Pág. 41) | Apuntes de Clase

Es fundamental no confundir los diferentes niveles y tipos de requerimientos que coexisten en un proyecto de software:



Jerarquía entre el Dominio del Problema y el Dominio de la Solución

1. DOMINIO DEL PROBLEMA (EL NEGOCIO Y LAS PERSONAS):

- **Requerimiento de Negocio:** Define las metas estratégicas de la organización que financia el proyecto.
Ejemplo de Cátedra: «Disminuir un 30% el tiempo invertido en procesos manuales de atención al cliente».
- **Requerimiento de Usuario:** Expresa las necesidades puntuales de las personas que interactúan con el sistema para materializar los requerimientos de negocio. **Aquí es donde viven las User Stories.**
Ejemplo de Cátedra: «Realizar consultas en línea del estado de cuenta de los clientes desde la app móvil».

2. DOMINIO DE LA SOLUCIÓN (EL SOFTWARE Y LA TECNOLOGÍA):

- **Requerimiento Funcional (RF):** Describe el comportamiento interno, servicios y cálculos que el sistema debe ejecutar.
Ejemplo de Cátedra: «Generar reporte consolidado de saldos en formato PDF y disparar notificaciones automáticas por correo electrónico».
- **Requerimiento No Funcional (RNF):** Establece restricciones de calidad, rendimiento, seguridad y disponibilidad.
Ejemplo de Cátedra: «El tiempo de respuesta de la consulta no debe exceder 2 segundos; el almacenamiento debe cumplir los estándares de seguridad de datos bancarios ley 9999XX».
- **Requerimiento de Implementación / Arquitectura:** Define decisiones tecnológicas y de infraestructura.
Ejemplo de Cátedra: «Despliegue sobre arquitectura de microservicios en la nube utilizando AWS ECS y base de datos PostgreSQL».

Principios Ágiles Vinculados a la Gestión de Requerimientos

Fuentes de Cátedra: Filmina 03 (Diaps. 29–31) | Resumen Teórico Final (Pág. 42) | Apuntes de Clase



Resumen visual de los principios aplicados a requerimientos

MÁXIMAS DE CÁTEDRA PARA EXAMEN:

1. **Los cambios son la única constante:** Pretender congelar requerimientos es negar la naturaleza del software.
2. **Stakeholders: no son todos los que están, ni están todos los que son:** Es crítico involucrar a los usuarios reales y no solo a intermediarios o gerentes lejanos a la operación diaria.
3. **El usuario dice lo que quiere cuando recibe lo que pidió:** La tangibilidad de un incremento funcional es el único catalizador que activa la comprensión real del cliente sobre sus verdaderas necesidades.
4. **No hay técnicas ni herramientas universales:** Toda práctica debe adaptarse al contexto específico del proyecto (*Tailoring*).
5. **Lo importante no es entregar código sin fallas (output), sino resolver el problema de negocio (outcome).**

User Stories (Historias de Usuario)

Fuentes de Cátedra: Filmina 04 (Diaps. 2–15) | Resumen Teórico Final (Págs. 42–44) | Ron Jeffries & Alistair Cockburn

Fuentes de Cátedra: Filmina 04 (Diaps. 2–15) | Resumen Teórico Final (Págs. 42–44) | Ron Jeffries & Alistair Cockburn

Origen y Problemática que Motivó la Técnica de User Stories

Fuentes de Cátedra: Filmina 04 (Diaps. 3–5) | Resumen Teórico Final (Pág. 42) | Extreme Programming (XP)

En la ingeniería de software tradicional, la definición de requerimientos ha sido históricamente la causa principal del fracaso de los proyectos. La problemática de fondo radica en la **dificultad intrínseca de comunicar con precisión lo que el sistema debe hacer a todas las personas afectadas** (usuarios, clientes, directivos, arquitectos, programadores y testers), considerando que cada parte interesada posee expectativas, intereses y lenguajes diferentes.

La elicitación tradicional (basada en extensas entrevistas iniciales y especificaciones contractuales SRS de cientos de páginas) no da buenos resultados debido a una falla epistemológica de raíz: **obliga a tomar todas las decisiones críticas al comienzo del proyecto, justo en el momento en que menos información y mayor incertidumbre se tiene**.

Frente a esto, el agilismo traslada la toma de decisiones a lo largo de todo el ciclo de vida del proyecto, a medida que la incertidumbre desciende y la información empírica aumenta gracias a la inspección de incrementos funcionales reales. Es en este contexto de descubrimiento continuo donde surgen las **User Stories (Historias de Usuario)**.

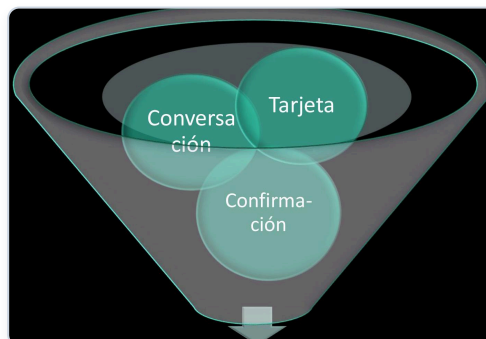
Definición y Partes de una User Story: Las 3C y Plantilla Canónica

Fuentes de Cátedra: Filmina 04 (Diaps. 6–15) | Resumen Teórico Final (Págs. 42–44) | Ron Jeffries (Card, Conversation, Confirmation)

Una **User Story** es una descripción breve y en lenguaje coloquial de una funcionalidad de software, relatada desde la perspectiva del usuario final o cliente que la necesita. Su objetivo no es documentar de forma exhaustiva, sino actuar como disparador para una conversación continua.

LAS 3 C DE UNA USER STORY (MODELO CLÁSICO DE RON JEFFRIES - XP):

Toda Historia de Usuario integral está constituida por tres componentes esenciales e inseparables:

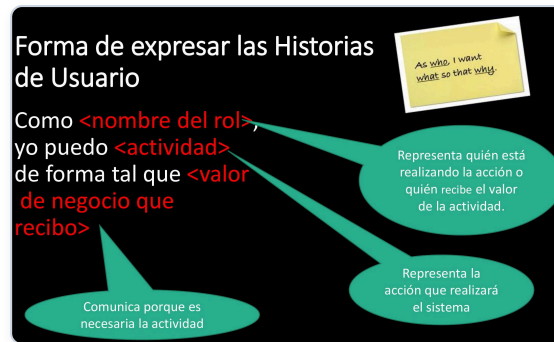


Tarjeta, Conversación y Confirmación (Ron Jeffries)

1. TARJETA (CARD):

- Es la representación física o digital sintética de la historia. Funciona como un **token o recordatorio para planificar y recordar una necesidad pendiente de discusión**.
- Debe ser deliberadamente pequeña y concisa (cabere en una tarjeta de cartulina o post-it).

Sintaxis formal de la Tarjeta (Plantilla estándar de Connextra):



Como [Rol], Quiero [Acción], Para [Beneficio]

- **Frase Verbal (Título corto):** No es obligatoria, pero se recomienda fuertemente para referenciar y rastrear la historia de forma rápida en el Product Backlog y tableros (ej.: «Buscar Destino por Dirección»).
- **Como <>:** Representa quién está realizando la acción o quién recibe directamente el beneficio de la funcionalidad.
- **Yo puedo / Quiero <>:** Representa la acción concreta que el usuario ejecutará con el sistema.
- **De forma tal que / Para <>:** Es la parte **MÁS IMPORTANTE** de la User Story. Representa el *por qué* es necesaria la actividad. Justifica plenamente la inversión del negocio, fundamenta por qué el Product Owner la prioriza en el Product Backlog y evita construir características innecesarias.
- *Ejemplo canónico de Cátedra:* «**Buscar Destino por Dirección:** Como **Conductor**, yo quiero **buscar un destino a partir de una calle y altura**, de forma tal que **pueda llegar al lugar deseado sin perderme**».

2. CONVERSACIÓN (CONVERSATION):

- Es la parte **MÁS IMPORTANTE** de la técnica de User Stories.
- Explicita la comunicación directa cara a cara entre el Product Owner y el Equipo de Desarrollo, compensando conscientemente la falta de especificación exhaustiva escrita.
- Representa el valor ágil de «*Colaboración con el cliente sobre negociación contractual*» y el principio de «*Técnicos y no técnicos trabajando juntos día a día*».
- El Product Owner forma parte integral del equipo; se requiere compromiso, responsabilidad y confianza mutua para alcanzar consensos sobre el alcance real de la funcionalidad.
- **¿Cómo persiste la conversación?** Puede registrarse en minutas, grabaciones o herramientas digitales de gestión (Jira, Azure DevOps, Trello), pero su materialización viva se refleja de forma distribuida en la tarjeta y en las pruebas de aceptación.

3. CONFIRMACIÓN (CONFIRMATION):

- Es el acuerdo formal entre el Product Owner y el Equipo de Desarrollo para demostrar objetivamente que la característica del producto fue implementada con éxito.
- Conformar la definición de las **pruebas de aceptación** (escritas típicamente al dorso de la tarjeta).
- El Product Owner es quien finalmente valida si las pruebas de aceptación son suficientes y decide si la funcionalidad se acepta para pasar a producción.

Ventajas y Desventajas de las User Stories

Fuentes de Cátedra: Filmina 04 (Diaps. 16–18) | Resumen Teórico Final (Págs. 44–45) | Apuntes de Clase

Fuentes de Cátedra: Filmina 04 (Diaps. 16–18) | Resumen Teórico Final (Págs. 44–45) | Apuntes de Clase

Un balance técnico indispensable evaluado en la página 44 y 45 del resumen histórico:

VENTAJAS PRINCIPALES:

- **Priorizan la comunicación verbal cara a cara:** Fomentan la interacción humana rica en matices por sobre la fría documentación escrita.
- **Lenguaje común y comprensible:** Se redactan en términos de negocio, resultando transparentes y entendibles tanto para el cliente/PO como para los desarrolladores y testers.
- **Tienen el tamaño justo para planificar:** Poseen la granularidad adecuada para estimar con agilidad y distribuir el trabajo en iteraciones cortas.
- **Facilitan el desarrollo iterativo e incremental:** Permiten dividir un producto complejo en rebanadas independientes que entregan valor temprano.
- **Desestiman detalles prematuros:** Aplican el principio *Just-in-Time*, posponiendo el análisis fino hasta el momento en que se tenga la información madura y necesaria.

DESVENTAJAS Y DESAFÍOS:

- **Dificultad de visión global en proyectos masivos:** Al ser historias independientes y atómicas, en proyectos que contienen cientos de Historias de Usuario puede resultar complejo comprender la arquitectura holística y las interrelaciones globales del sistema.
- **Inadecuadas para equipos gigantes o desarticulados:** Si el equipo es demasiado grande o carece de canales directos de conversación diaria con el PO, el modelo de historias puede fallar, requiriendo artefactos de coordinación adicionales.

Las User Stories son Multipropósito

Fuentes de Cátedra: Filmina 04 (Diaps. 19–20) | Resumen Teórico Final (Pág. 45) | Apuntes de Clase

Una Historia de Usuario desempeña cinco funciones simultáneas dentro del marco ágil:

- Es la expresión condensada de una **necesidad real del usuario**.
- Es una **descripción incremental del producto** a construir.
- Es un **ítem cuantificable de planificación y estimación** (PBI).
- Es un **token o disparador para una conversación** rica entre el negocio y la técnica.
- Es un **mecanismo para diferir el análisis detallado** hasta el momento oportuno (evitando el desperdicio temprano).

Propósito Esencial: Identificar Requerimientos de Usuario, NO Especificar Software

Fuentes de Cátedra: Filmina 04 (Diap. 20) | Resumen Teórico Final (Pág. 41) | Apuntes de Clase

- **No son especificaciones detalladas ni casos de uso extensos:** Los casos de uso tradicionales intentan capturar todos los caminos posibles de forma anticipada. Las historias de usuario son expresiones de intención (*«necesitamos lograr algo como esto...»*), otorgando libertad al equipo para diseñar la mejor solución técnica.
- **Se elaboran progresivamente:** Se refinan a medida que suben en la pila del Product Backlog, evitando el inventario estéril de requisitos.
- **Bajo costo de mantenimiento y descartabilidad:** Una vez que la historia se programa, aprueba las pruebas de aceptación y se integra al incremento, la tarjeta física puede descartarse. La verdadera «documentación viva» pasa a ser el **código fuente limpio y su suite de pruebas automatizadas**.

PRINCIPIO FUNDAMENTAL DE CÁTEDRA: USER STORIES VS. ESPECIFICACIÓN DE SOFTWARE:

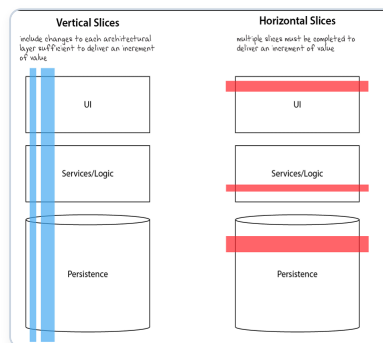
Las User Stories NO sirven para especificar requerimientos de software; sirven para IDENTIFICAR requerimientos de usuario.

- El foco de la gestión ágil está puesto directamente sobre el **usuario final y sus dolores u objetivos**.
- En marcos como Scrum, quien trabaja primordialmente para identificar las User Stories es el **Product Owner**, ya que es el responsable de entender las necesidades de los usuarios para cumplir los objetivos estratégicos del negocio.
- El equipo de desarrollo es quien luego diseña la solución técnica (base de datos, arquitectura, APIs) que materializa y da soporte a esa necesidad de usuario.

User Stories como Porciones Verticales a través de la Arquitectura

Fuentes de Cátedra: Filmina 04 (Diaps. 21–25) | Resumen Teórico Final (Págs. 45–46) | Apuntes de Clase

Fuentes de Cátedra: Filmina 04 (Diaps. 21–25) | Resumen Teórico Final (Págs. 45–46) | Apuntes de Clase



Principio de la porción vertical (Cake Slice): Una historia atraviesa todas las capas arquitectónicas

El Principio de la Rebanada de Pastel (Vertical Slice) y su Relación con la Arquitectura en Capas:

- Para agregar valor genuino al cliente, una User Story debe concebirse como una **porción vertical completa**.
- Debemos diseñar y construir el pedacito de cada capa tecnológica que hace falta para que esa funcionalidad específica funcione de punta a punta: un poco de **Interfaz de Usuario (GUI)**, un poco de **Lógica de Negocio / Servicios** y un poco de **Base de Datos / Persistencia**.
- **Regla Estricta de Cátedra:** NUNCA se deben formular historias de usuario horizontales (ej.: «Diseñar la base de datos de usuarios», «Construir los endpoints de la API» o «Maquetar el frontend»). Una capa horizontal aislada no es integrable de forma independiente, no entrega valor perceptible al usuario y no permite validar empíricamente el progreso del producto.

Modelado de Roles, Personas y Usuarios Representantes (Proxies)

Fuentes de Cátedra: Filmina 04 (Diaps. 26–35) | Resumen Teórico Final (Pág. 47) | Alan Cooper & Mike Cohn

Fuentes de Cátedra: Filmina 04 (Diaps. 26–35) | Resumen Teórico Final (Pág. 47) | Alan Cooper & Mike Cohn

El aspecto crítico de cualquier funcionalidad radica en **quién la va a utilizar**. En Agile existe una disciplina explícita para modelar a los usuarios mediante **Tarjetas de Rol de Usuario**, donde se definen las características y el perfil genérico de las personas frente al producto.

TÉCNICAS ADICIONALES DE MODELADO DE USUARIOS:

1. Personas:

- Esta técnica busca ser mucho más específica que un rol genérico. Consiste en crear una descripción biográfica profunda de una persona concreta y ficticia que encarna al arquetipo de usuario (nombre, fotografía, edad, hábitos, contexto laboral y frustraciones).
- *Metáfora de Cátedra*: Si los **Roles son clases**, las **Personas son objetos instanciados**.

2. Personajes Extremos:

- Consiste en analizar qué usuarios situados en los extremos del espectro podrían utilizar el sistema (ej.: un usuario con nula alfabetización digital vs. un experto informático; un anciano con visión disminuida vs. un conductor que opera la app en un entorno ruidoso).
- Ayuda a identificar requerimientos de accesibilidad, seguridad y ergonomía que enriquecen a las User Stories.

3. Usuarios Representantes (Proxies):

- Surgen cuando el Product Owner o usuario ideal de la trinchera no puede participar diariamente en el proyecto.
- Un Proxy es un representante de los usuarios que asume la responsabilidad de priorizar y tomar decisiones de negocio.
- **Ejemplos típicos**: Gerentes de usuarios, gerentes de desarrollo, personal de marketing, vendedores, expertos del dominio (SMEs) y personal de soporte técnico.
- ⚠ *Alerta de Cátedra*: Los Proxies son un «mal necesario» a evitar si es posible. Al no estar en el día a día operativo, pueden introducir sesgos, distorsionar necesidades y ralentizar las respuestas.

Criterios de Aceptación de User Stories

Fuentes de Cátedra: Filmina 04 (Diaps. 36–42) | Resumen Teórico Final (Págs. 47–48) | BDD (Given-When-Then)

Los **Criterios de Aceptación** son acuerdos explícitos respecto a cómo debe comportarse el software para que el Product Owner acepte la implementación. Son indispensables porque proporcionan a los desarrolladores y testers un parámetro objetivo para saber si la funcionalidad fue cumplimentada con éxito.

PROPÓSITOS FUNDAMENTALES:

- **Definen los límites (boundaries) de la User Story**: Indican a los desarrolladores con precisión **cuándo parar de agregar funcionalidad**, previniendo el *gold-plating*.
- Ayudan al Product Owner a explicitar los requerimientos funcionales mínimos para que la historia provea valor real.
- Unifican la visión técnica y de negocio sobre la historia.
- Constituyen la base directa para que desarrolladores y testers diseñen las pruebas.

¿CÓMO SON LOS BUENOS CRITERIOS DE ACEPTACIÓN?

- **Definen una intención, no una solución (el QUÉ, no el CÓMO)**:
Ejemplo Correcto: «El usuario debe seleccionar al menos una cuenta habilitada para operar».
Ejemplo Incorrecto: «El sistema ejecuta una sentencia SELECT contra la tabla CUENTAS usando un combo desplegable».
- **Son independientes de la tecnología de implementación.**
- **Se mantienen en un nivel de abstracción relativamente alto.**

¿Y LOS DETALLES FINOS? ¿A DÓNDE VAN?

Detalles específicos como: el encabezado exacto de una columna («Saldo»), el formato numérico («999.999.999,99») o la preferencia por una lista desplegable en lugar de casillas de verificación **NO deben**

ensuciar los criterios principales de la tarjeta. El equipo los captura en:

1. Documentación interna y wireframes del equipo de desarrollo.
2. **Pruebas de aceptación automatizadas** (escritas en herramientas como Cucumber, Behave o SpecFlow).

Pruebas de Aceptación de Historias de Usuario

Fuentes de Cátedra: Filmina 04 (Diaps. 43–45) | Resumen Teórico Final (Pág. 48) | Cátedra ISW

Las pruebas de aceptación son declaraciones de intención sobre qué debe validarse en el sistema. Expresan los detalles resultantes de las conversaciones entre el PO y el equipo. **El Product Owner acepta la User Story como terminada si y solo si todas las pruebas de aceptación se cumplen con éxito.**

- Deben escribirse **antes de que comience la programación** de la historia (alineadas con la filosofía TDD / ATDD).
- Contemplan tanto escenarios de éxito (caminos felices) como escenarios de fallo y casos de borde.
- Se estructuran en dos pasos: 1) Identificación de escenarios en el dorso de la tarjeta, 2) Diseño y automatización de las pruebas ejecutables.

EJEMPLO DE CÁTEDRA: TARJETA Y CASOS DE PRUEBA:

Historia: *«Como Conductor quiero buscar un destino a partir de sus coordenadas para conocer el camino a recorrer para llegar al destino deseado».*

Pruebas de Aceptación Asociadas:

- **Prueba de Flujo Positivo:** Probar buscar un destino en un país y ciudad existentes, a partir de dos coordenadas válidas existentes → El sistema traza y muestra la ruta con éxito (PASA).
- **Prueba de Caso Negativo / Borde:** Probar buscar un destino con coordenadas en zona oceánica o fuera de límites cartografiados → El sistema emite una advertencia clara sin romperse (PASA).

Definición de Listo (DoR) y Definición de Hecho (DoD): Compuertas de Calidad

Fuentes de Cátedra: Filmina 04 (Diaps. 46–55) | Resumen Teórico Final (Págs. 48–49) | Guía Scrum & Roman Pichler

La cátedra hace un énfasis primordial en diferenciar estos dos acuerdos de calidad que operan como compuertas en el flujo del trabajo ágil:

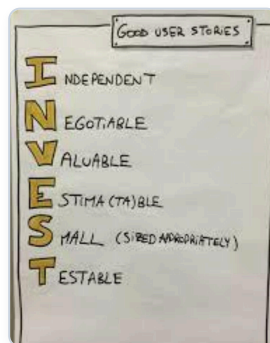
Eje de Comparación	Definition of Ready (DoR) — Definición de Listo	Definition of Done (DoD) — Definición de Hecho
Momento y Función	Compuerta de ENTRADA al Sprint Determina si un PBI está maduro para ser planificado y admitido en una iteración de desarrollo.	Compuerta de SALIDA del Sprint Determina si una User Story y el Incremento están terminados y en condiciones de pasar a producción.
Nivel de Aplicación	Se evalúa de forma individual sobre cada User Story .	Aplica de forma transversal a todo el equipo, a todas las historias y al Incremento completo .
Criterio Mínimo Exigido	Para entrar al Sprint, la historia mínimamente debe satisfacer el Modelo INVEST . El equipo y el PO pueden acordar condiciones adicionales.	Debe cumplir con el Checklist integral de calidad acordado por el equipo o la organización.
Aporte a los Pilares Ágiles	Evita el desperdicio de iniciar trabajo con requerimientos caóticos o bloqueados por dependencias externas.	Genera Transparencia: Proporciona a todo el equipo y a los interesados una comprensión exhaustiva de qué trabajo se realizó para lograr ese incremento.
Contenido Típico del Checklist	• Cumple estrictamente con el modelo INVEST. • Criterios de Aceptación claros y acordados con el PO. • Historia debidamente estimada en Story Points. • Dependencias técnicas o externas resueltas. • Wireframes o definiciones de negocio disponibles.	• Código revisado por pares (Pull Request aprobado). • Pruebas unitarias y de integración pasando al 100%. • Criterios de Aceptación verificados y aprobados por el PO. • Cero defectos críticos abiertos. • Desplegado automáticamente en ambiente de Staging. • Documentación técnica y manuales actualizados.

INVEST Model y la Regla del 13

Fuentes de Cátedra: Filmina 04 (Diaps. 56–65) | Resumen Teórico Final (Pág. 48) | Bill Wake & Mike Cohn

Fuentes de Cátedra: Filmina 04 (Diaps. 56–65) | Resumen Teórico Final (Pág. 48) | Bill Wake & Mike Cohn

El modelo **INVEST** (propuesto por Bill Wake) es el estándar de calidad fundamental que determina si una User Story cuenta con las características mínimas para satisfacer la **Definition of Ready (DoR)** e ingresar a un Sprint:



Los seis atributos de calidad del modelo INVEST (Bill Wake)

1. I — Independent (Independiente):

- La User Story es calendarizable e implementable en cualquier orden.

- No debe existir una dependencia técnica rígida que obligue a construir una historia para poder iniciar o mostrar otra.
- Otorga total libertad al Product Owner para priorizar las historias en el Product Backlog según las conveniencias cambiantes del mercado.

2. N — Negotiable (Negociable):

- Las historias no son contratos cerrados ni especificaciones inmutables.
- La tarjeta actúa como recordatorio de una necesidad abierta a la conversación futura.
- El cliente/PO expresa el «QUÉ» necesita, mientras que el «CÓMO» es decisión del equipo técnico y debe ser negociable para buscar la solución más eficiente.

3. V — Valuable (Valiosa):

- Una User Story debe aportar valor perceptible de negocio al cliente o usuario final (el «*Para qué*» de la tarjeta).
- No se escriben historias que solo tengan valor interno para los programadores (ej.: no se escribe una historia para «Configurar el servidor web» o «Instalar una librería»); las tareas técnicas se integran como parte del trabajo de una historia que aporte valor funcional.

4. E — Estimatable (Estimable):

- La historia debe contener información suficiente para que el equipo estime su tamaño relativo (Story Points), complejidad y esfuerzo.
- Sin poder estimarla, no es posible saber si cabe en una iteración ni proyectar la duración del release.
- 🚩 **Las 3 Razones por las que una User Story NO es estimable (Pregunta de Examen):**
 - 1. *El equipo tiene incertidumbre acerca del dominio del negocio*: Solución → Dialogar y reunirse con el Product Owner para esclarecer todas las dudas.
 - 2. *El equipo tiene incertidumbre técnica (dominio de la solución)*: Solución → Destinar a uno o más miembros del equipo a realizar uno o más **Spikes Técnicos**.
 - 3. *La historia es demasiado grande (Épica)*: Solución → Descomponer la historia en User Stories más pequeñas.

5. S — Small (Pequeña):

- La historia debe ser lo suficientemente pequeña como para ser comenzada y finalizada dentro de una única iteración (Sprint). No deben quedar trabajos a medias.
- **El tamaño es relativo**: Depende de la experiencia del equipo, de su capacidad de trabajo y de la duración fija de la iteración.
- **Gestión Binaria**: El agilismo erradica el **Síndrome del 90-90** (*«el primer 90% del código toma el 90% del tiempo; el 10% restante toma el otro 90% del tiempo»*). En Agile no se gestiona por porcentaje de avance: una historia está 100% terminada (según DoD) o tiene 0% de valor.

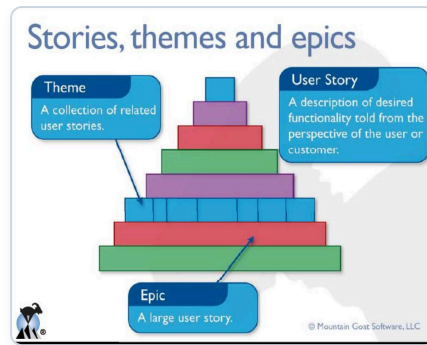
6. T — Testable (Testable / Comprobable):

- La historia debe contar con criterios y pruebas de aceptación objetivos que permitan demostrar formalmente si fue implementada de forma correcta y completa.

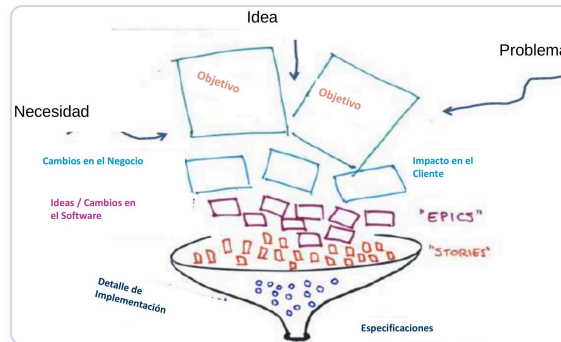
Diferentes Niveles de Abstracción: Temas, Épicas, Features, User Stories y Tareas

Fuentes de Cátedra: Filmina 04 (Diaps. 66–70) | Resumen Teórico Final (Págs. 46–47) | Apuntes de Clase

Para reducir los esfuerzos cognitivos de estimación y planificación, los requerimientos se gestionan en tres niveles de granularidad progresiva:



Jerarquía: Theme → Epic → User Story



Descomposición progresiva de Temas en Épicas e Historias de Usuario

Nivel	Definición Formal	Alcance Temporal / Características	Ejemplo de Cátedra
Tema (Theme)	Conjunto de User Stories o iniciativas agrupadas conceptualmente debido a que están estrechamente relacionadas, tratándose como una entidad simple a la hora de estimar a largo plazo.	Varios meses, trimestres o releases de producto.	«Optimización integral de la experiencia de pagos y facturación».
Épica (Epic)	Es una User Story muy grande. El tamaño es relativo a si cabe o no en una iteración: si no entra en un Sprint o no cumple con INVEST, se clasifica como Épica. Comparte la misma sintaxis que la User Story. Permite gestionar requerimientos que no son prioritarios en las primeras iteraciones.	Varios Sprints sucesivos (1 a 3 meses). Se descompone en historias pequeñas cuando sube en la pila del backlog.	«Implementar módulo de cobro mediante billeteras virtuales y códigos QR».
User Story (US)	Unidad atómica de funcionalidad con valor de negocio independiente. Cumple estrictamente con la Definition of Ready (DoR) y el modelo INVEST para ingresar a un Sprint.	Días de trabajo (cabe cómodamente dentro de un único Sprint).	«Como cliente registrado, quiero pagar mediante código QR para completar mi compra sin usar tarjeta física».

INVESTMENT THEMES (TEMAS DE INVERSIÓN):

En el nivel corporativo (página 50-51 del resumen histórico), los **Investment Themes** conducen las decisiones estratégicas de inversión en sistemas para lograr diferenciación en el mercado. Se dividen en 5 categorías fundamentales:

1. **Inversión en ofertas de productos existentes:** Mejoras continuas, soporte operativo y mantenimiento evolutivo.

2. **Inversión en nuevos productos y servicios:** Iniciativas para capturar cuota de mercado y generar beneficios en el corto plazo.
3. **Inversión a futuro en productos y servicios avanzados:** Proyectos de I+D e innovación tecnológica disruptiva.
4. **Inversión hoy con retorno a largo plazo:** Apuestas estructurales cuyos beneficios se materializarán en varios años.
5. **Inversión en reducción (Estrategia de Retiro / Sunset):** Gestión ordenada del fin de vida útil de sistemas legados u obsoletos.

Spikes Técnicos vs. Spikes Funcionales

Fuentes de Cátedra: Filmina 04 (Diaps. 71–73) | Resumen Teórico Final (Págs. 49–50) | Extreme Programming (XP)

Fuentes de Cátedra: Filmina 04 (Diaps. 71–73) | Resumen Teórico Final (Págs. 49–50) | Extreme Programming (XP)

Un **Spike** es un tipo especial de User Story que surge ante la presencia de una **incertidumbre extrema que imposibilita la estimación** y bloquea el cumplimiento de la Definition of Ready. Es una característica más del producto a la cual el equipo le dedica tiempo formal de trabajo. **Una vez resuelta la incertidumbre, el Spike desaparece y se convierte en una o más User Stories estimables.**

CLASIFICACIÓN DE SPIKES:

1. Spikes Técnicos:

- Se enfocan en evaluar diferentes enfoques técnicos y tecnológicos en el **dominio de la solución**.
- Asociados a la implementación: investigar una nueva librería, evaluar rendimiento bajo carga o analizar decisiones de arquitectura *Make or Buy*.
- Dependen exclusivamente de los criterios de los técnicos (el equipo de desarrollo).

2. Spikes Funcionales:

- Se enfocan en el **dominio del problema**.
- Se utilizan cuando hay incertidumbre respecto a cómo el usuario interactuará con el sistema para alcanzar el beneficio deseado.
- Se resuelven típicamente mediante **prototipos rápidos para obtener retroalimentación directa de los usuarios involucrados**.
- Dependen de definiciones de negocio y de la interacción con el Product Owner.

Lineamientos Rigurosos para Spikes: La Excepción, no la Regla

Fuentes de Cátedra: Filmina 04 (Diaps. 74–75) | Resumen Teórico Final (Pág. 50) | Cátedra ISW

Fuentes de Cátedra: Filmina 04 (Diaps. 74–75) | Resumen Teórico Final (Pág. 50) | Cátedra ISW

Algunas cosas para dejar en claro



DIFERIR EL ANÁLISIS DETALLADO TAN TARDE COMO SEA POSIBLE, LO QUE ES JUSTO ANTES DE QUE EL TRABAJO COMIENCE.



HASTA ENTONCES, SE CAPTURAN REQUERIMIENTOS EN LA FORMA DE "USER STORIES".



LAS USER STORIES NO SON REQUERIMIENTOS DE SOFTWARE, NO NECESITAN SER DESCRIPCIONES EXHAUSTIVAS DE LA FUNCIONALIDAD DEL SISTEMA.

Gestión temporal y timebox de Spikes

Tips para que las user stories sean útiles para el equipo



Un paso a la vez (evitar la palabra "Y")



Usar palabras claras en los criterios de aceptación



No olvides la parte invisible: la conversación



Las user stories se escriben desde la perspectiva del usuario



No forzar todo para escribirlo como user stories

Diferenciación entre Spikes e Historias productivas

1. **Son estimables, demostrables y aceptables:** El entregable de un Spike no es software productivo, sino **conocimiento validado** (un informe técnico, una prueba de concepto o un prototipo).
2. **Son estrictamente Timeboxed:** Se les asigna un límite fijo de horas o días de investigación. Al cumplirse el plazo, se cierra el Spike con los hallazgos obtenidos.
3. **La excepción, no la regla:** Toda historia ágil convive con cierto grado de incertidumbre. El equipo debe aprender a convivir y resolver la incertidumbre cotidiana en cada Sprint. Los Spikes se reservan como **última opción** para incógnitas críticas y de alto impacto.
4. **Se implementan en una iteración separada de las historias resultantes:** El Spike se ejecuta en el Sprint actual para habilitar la estimación de las historias que se construirán en el Sprint siguiente (salvo que sea un Spike muy pequeño que pueda resolverse en pocas horas dentro de la misma iteración).

🔥 ¡Clave de Parcial! Malos Olores en User Stories (¿Qué huele mal?)

En las filminas de cátedra (diapositivas 62 a 65) se analizan ejemplos típicos de examen sobre errores graves al formular historias:

1. ❌ «El software deberá entregarse el 30 de Julio»

→ ¿Por qué está mal? No es una necesidad de usuario; es una **restricción de cronograma y un compromiso de gestión**.

2. ❌ «El software deberá estar desarrollado en JAVA / utilizar Log4J para registrar errores / base de datos Oracle»

→ ¿Por qué está mal? Impone la solución tecnológica (el CÓMO) en la tarjeta. Viola la negociabilidad y distorsiona el foco en el valor del negocio (el QUÉ).

3. ❌ «El usuario podrá seleccionar su país desde una lista desplegable»

→ ¿Por qué está mal? Prescribe el componente de UI de forma anticipada. Los detalles de interfaz pertenecen al diseño y a la conversación, no a la definición de la necesidad.

4. ❌ «Como usuario quiero registrarme Y subir una foto Y configurar mis preferencias de cobro»

→ ¿Por qué está mal? Contiene la conjunción «Y». Viola el principio **Small** de INVEST; es una Épica que agrupa múltiples funcionalidades independientes y debe desglosarse en historias atómicas.

Investment Themes (Temas de Inversión): Alineación Estratégica y Presupuestaria

Fuentes de Cátedra: Filmina 04 (Diaps. 76–78) | Resumen Teórico Final (Págs. 50–51) | Mike Cohn

Los **Investment Themes (Temas de Inversión)** representan las áreas de iniciativa estratégica de más alto nivel a través de las cuales una organización decide distribuir y presupuestar su capacidad de ingeniería de software:

- Permiten a los directores de negocio y al Product Owner definir qué porcentaje de la capacidad del equipo se destinará a diferentes propósitos corporativos.
- Un Tema de Inversión agrupa a múltiples Épicas y Features a lo largo de varios trimestres (*Roadmap*).

LAS CINCO CATEGORÍAS CANÓNICAS DE ASIGNACIÓN ESTRATÉGICA DE INVERSIÓN (RESUMEN FINAL, PÁGS. 50–51):

1. **Inversión en Nuevas Características para Clientes Existentes:** Orientada a profundizar la funcionalidad del producto, fidelizar la base actual de usuarios y mejorar la satisfacción general.
2. **Inversión en Nuevos Mercados o Segmentos de Clientes:** Enfocada en construir capacidades requeridas para capturar nuevos perfiles de usuario, expandirse a nuevas regiones geográficas o abordar nuevos modelos de negocio.
3. **Inversión en Infraestructura Técnica y Reducción de Deuda Técnica:** Dirigida a refactorización, modernización de arquitectura, seguridad, actualización de frameworks y escalabilidad. No aporta funcionalidades visibles al usuario final, pero es vital para mantener la velocidad sostenible del equipo.
4. **Inversión a Largo Plazo con Retorno Diferido (I+D e Innovación):** Apuestas estratégicas que no darán beneficios inmediatos sino dentro de 2 o 3 años (pruebas de concepto con tecnologías disruptivas, algoritmos experimentales).
5. **Inversión en Reducción y Retiro Paulatino (Estrategia de Obsolescencia):** Recursos asignados para dismantelar, migrar o apagar sistemas legados que están cerca del fin de su ciclo de vida, minimizando riesgos operativos de transición.

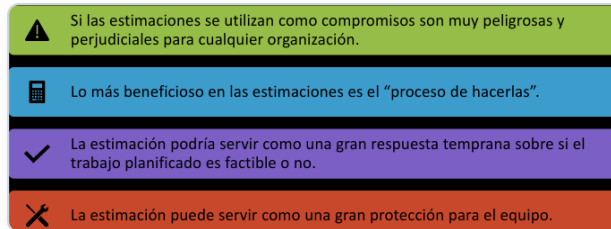
Tips de Cátedra y Detección de Bad Smells en Historias de Usuario

Fuentes de Cátedra: Filmina 04 (Diap. 71) | Resumen Teórico Final (Págs. 44–46) | Cátedra ISW

- **Un paso a la vez (evitar la palabra «Y»):** Si una historia hace más de una cosa, divídela.
- **Usar palabras claras en los criterios de aceptación:** Evitar ambigüedades subjetivas como «rápido», «fácil» o «intuitivo».
- **No olvidar la parte invisible: la conversación:** La tarjeta es solo el disparador; el verdadero valor emerge del diálogo cara a cara.
- **Escribir siempre desde la perspectiva del usuario:** La historia debe reflejar una necesidad viva de quien usa el sistema, no un requerimiento de sistema abstracto.
- **No forzar todo para que sea una User Story:** Las restricciones globales de arquitectura, seguridad o normativas no deben disfrazarse forzosamente de historias de usuario; pertenecen a los Requerimientos No Funcionales o forman parte de la Definition of Done (DoD).

Estimaciones en Ambientes Ágiles

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 2–8) | Resumen Teórico Final (Págs. 51–53) | Steve McConnell & Mike Cohn



El universo de la estimación: probabilidad e incertidumbre

Cualquier proceso de estimación en software opera dentro de un universo gobernado por la **incertidumbre y la probabilidad**. Lo contrario a la estimación es la certeza absoluta, la cual solo existe cuando el software ya fue completamente construido y entregado.

PRINCIPIOS RECTORES DE LA CÁTEDRA:

- **No ser dogmático sobre nada:** Ser pragmático es la clave del éxito. Las decisiones de estimación deben ajustarse al contexto de la organización, a la madurez del equipo y al momento del proyecto.
- **La estimación como herramienta de protección:** Una estimación honesta protege al equipo de compromisos delirantes y sirve como alarma temprana sobre la viabilidad de lo planificado.
- **Regla de Oro Institucional:**
«Las estimaciones **NUNCA** se negocian. Se negocian los compromisos».
Presionar a un equipo para que reduzca artificialmente un número de estimación no reduce el trabajo a realizar; únicamente destruye la transparencia y genera deuda técnica.

Naturaleza de los Errores en las Estimaciones

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 2-4) | Resumen Teórico Final (Págs. 51-52) | Steve McConnell

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 2-4) | Resumen Teórico Final (Págs. 51-52) | Steve McConnell

Las 7 causas fundamentales de desvío y fracaso en la estimación de software:

1. **Información imprecisa:** Desconocimiento sobre el producto a construir o sobre las capacidades reales del equipo ejecutor.
2. **Caos en el proyecto:** Indefinición del alcance, requisitos ambiguos y falta de liderazgo de producto.
3. **Imprecisión del proceso de estimación:** Utilizar técnicas inadecuadas o carecer de criterios compartidos de medición.
4. **Omisión de actividades esenciales:** Olvidar estimar tareas críticas e insoslayables como reuniones de coordinación, revisiones técnicas, gestión de licencias, armado de ambientes y pruebas de regresión.
5. **Confundir planear con estimar:** La estimación es la base y el insumo de los planes, pero los planes incorporan holguras, contingencias y asignación de recursos. Planificar no es estimar.
6. **Brecha entre lo planeado y lo estimado:** A mayor diferencia entre las estimaciones técnicas y los planes impuestos, mayor es el riesgo de colapso.
7. **Tratar las estimaciones como compromisos contractuales inamovibles.**

Matriz de Desambiguación: Estimación vs. Objetivo vs. Compromiso

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diap. 5) | Resumen Teórico Final (Pág. 54) | Steve McConnell

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diap. 5) | Resumen Teórico Final (Pág. 54) | Steve McConnell

Es un requisito conceptual estricto de la ingeniería de software distinguir con total claridad estos tres conceptos:

Concepto	Término en Inglés	Definición y Características	Naturaleza
Objetivo	Target	Es una meta de negocio deseada impuesta por los interesados o la dirección comercial. <i>Ejemplo:</i> «Lanzar la plataforma antes de Navidad para capturar el mercado de fin de año».	Subjetivo. Representa lo que a la organización le gustaría que suceda.
Estimación	Estimate	Es una predicción analítica, neutral e imparcial del tamaño, esfuerzo o costo requerido para realizar un trabajo. <i>Ejemplo:</i> «Calculamos que este conjunto de historias requerirá entre 45 y 55 Story Points con un 85% de probabilidad».	Objetivo y probabilístico. Se sustenta en parámetros empíricos. Nunca se negocia.
Compromiso	Commitment	Es una promesa explícita de entrega acordada con el cliente tras compatibilizar el objetivo de negocio con la estimación técnica y los riesgos analizados. <i>Ejemplo:</i> «Nos comprometemos a entregar las funcionalidades esenciales del MVP para el 15 de diciembre».	Acuerdo formal. Es lo que se negocia y gestiona a través de la planificación.

La Dinámica de los Procesos de Software y el Proceso de Estimación

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 6–8) | Resumen Teórico Final (Págs. 52–54) | Apuntes de Clase

El Círculo Vicioso de la Subestimación

«Una estimación artificialmente baja no acelera el desarrollo; simplemente recorta las fundaciones, garantizando que los errores se descubran en la fase más cara y crítica del proyecto».

- 1. Cronograma irreal dictado por el negocio:** Se impone una fecha arbitraria sin respaldo técnico.
 - 2. Recorte de fases tempranas:** Presionado por el tiempo, el equipo recorta actividades indispensables de análisis, diseño arquitectónico y pruebas.
 - 3. Soluciones rápidas y sucias:** Se codifica con pésimas prácticas y sin refactorizar (deuda técnica masiva).
 - 4. Altas tasas de defectos:** El sistema explota en fallos durante las pruebas integrales o directamente en producción.
 - 5. Crisis permanente:** Reuniones de emergencia, parches apresurados, horas extras y desvío de personal.
- **Resultado:** El proyecto termina costando mucho más y entregándose mucho más tarde que bajo una estimación honesta.

EL CONO DE LA INCERTIDUMBRE (STEVE MCCONNELL / BARRY BOEHM):

- Al inicio del proyecto (fase de concepto), el factor de incertidumbre oscila entre **0.25x y 4.0x** (un rango de error potencial de hasta 16 veces).
- Exigir estimaciones exactas en horas al principio es una quimera.
- A medida que se avanza en el proyecto, la incertidumbre desaparece progresivamente.
- La respuesta ágil:** La certidumbre no se logra haciendo planes teóricos más extensos, sino mediante **incrementos de software funcionando**. No podemos planificar para eliminar la incertidumbre; debemos iterar para descubrirla y resolverla empíricamente.

¿Cómo se Logra una Buena Estimación en Agile?

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 9–11) | Resumen Teórico Final (Pág. 54) | Apuntes de Clase

Una estimación profesional en ingeniería de software debe satisfacer cuatro condiciones indispensables:

1. **Estar expresada en rangos:** Evita la falsa exactitud puntual. Se expresa como un intervalo (ejemplo: «entre 12 y 16 semanas» en lugar de decir «14 semanas exactas»).
2. **Incluir un nivel de probabilidad explícito:** Todo rango debe declarar su confianza asociada (ejemplo: «12 a 16 semanas con un 80% de probabilidad de cumplimiento»).
3. **Estar totalmente desacoplada del objetivo de negocio:** La estimación técnica no debe distorsionarse ni «inflarse» o «desinflarse» para complacer las aspiraciones del cliente.
4. **Habilitar decisiones de control:** Debe ofrecer datos objetivos que permitan al negocio priorizar, ajustar el alcance o reasignar recursos oportunamente.

Tamaño vs. Esfuerzo: Estimaciones Relativas y Absolutas

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 12–14) | Resumen Teórico Final (Págs. 55, 57) | Mike Cohn

Las metodologías tradicionales y los marcos ágiles difieren radicalmente en su concepción de la medición:

1. ESTIMACIONES ABSOLUTAS (ENFOQUE TRADICIONAL):

- Intentan predecir el tiempo exacto en unidades lineales fijas (horas-hombre o días de calendario).
- **Falla estructural:** El esfuerzo es estrictamente individual. Una misma funcionalidad puede insumirle 2 horas a un desarrollador senior y 12 horas a un junior con poca experiencia.
- Los seres humanos somos notoriamente deficientes estimando tiempos absolutos en actividades intelectuales y creativas complejas.

2. ESTIMACIONES RELATIVAS (ENFOQUE ÁGIL):

- Se basan en la **comparación directa de magnitudes:** «Esta historia es el doble de grande o compleja que esta otra».
- La psicología cognitiva demuestra que los humanos somos notablemente rápidos y acertados comparando cosas en términos relativos: todos podemos distinguir de inmediato que un perro Gran Danés es mucho más grande que un Caniche, sin necesidad de saber con exactitud cuántos gramos pesa cada uno.
- Fomenta el pensamiento colectivo de equipo por sobre la subjetividad individual y agiliza el análisis de las historias.

DIFERENCIA FUNDAMENTAL: TAMAÑO VS. ESFUERZO

- **Tamaño:** Es la magnitud intrínseca del trabajo requerido para construir una historia. Es una propiedad del producto, objetiva e idéntica para todo el equipo, y se cuantifica en **Story Points (SP)**.
- **Esfuerzo:** Son las horas-persona lineales que a un individuo en particular le demandará construir ese tamaño, dependiendo de sus habilidades, herramientas y experiencia personal.

ESCALAS DE TAMAÑO RELATIVO:

1. **Números del 1 al 10.**
2. **Talles de remeras (XS, S, M, L, XL):** Ideal para estimaciones de alto nivel en el Product Backlog y para clasificar Épicas.
3. **Serie exponencial de base 2 (1, 2, 4, 8, 16, 32, 64, etc.).**

4. Serie de Fibonacci Modificada: Popularizada por Mike Cohn:

0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100

(A partir del 13, se redondea a 20, 40 y 100 para evitar dar una falsa ilusión de precisión; no tiene sentido discutir si algo vale 21 o 22 puntos cuando la incertidumbre es enorme).

⚠ El Principio del Metro Patrón

Una vez que el equipo elige una escala de estimación y define su historia canónica de referencia, **la escala NO se cambia**.

Modificar la escala equivale a cambiar la unidad de medida en mitad del proyecto, lo que destruiría la coherencia del histórico de datos y la medición de la velocidad del equipo.

¿Qué es un Story Point? Incertidumbre, Complejidad y Esfuerzo

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 15–20) | Resumen Teórico Final (Págs. 55, 57) | Serie de Fibonacci

Un **Story Point (Punto de Historia)** es una unidad de medida relativa y abstracta, específica de cada equipo, que cuantifica el **tamaño global** de una Historia de Usuario respecto a una unidad patrón tomada como referencia.

LAS TRES DIMENSIONES DEL STORY POINT:

$$\text{Story Point} = \text{Complejidad Técnica} + \text{Volumen de Esfuerzo} + \text{Incertidumbre / Riesgo}$$

1. **Complejidad Técnica:** Qué tan difícil, intrincada o novedosa es la solución lógica y técnica requerida.
2. **Volumen de Esfuerzo:** La cantidad neta de trabajo a realizar (independientemente de si es cognitivamente complejo o rutinario).
3. **Incertidumbre / Riesgo / Duda:** Cuánta información falta, la presencia de dependencias externas o el grado de desconocimiento del negocio o de la tecnología.

LA HISTORIA CANÓNICA (BASE STORY):

Para calibrar el sistema de medición, el equipo selecciona una historia sencilla y de mínimo riesgo que todos comprenden a la perfección y le asigna el valor base de **1 Story Point (o 2 SP)**. Esta historia opera como el «kilo patrón»: a partir de allí, todas las demás historias se estiman diciendo: «¿Cuántas veces más grande, compleja o riesgosa es esta historia respecto a nuestra historia base?».

Planning Poker (Estimación con Naipes) Paso a Paso

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 21–25) | Resumen Teórico Final (Págs. 58–59) | Mike Cohn & James Grenning

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 21–25) | Resumen Teórico Final (Págs. 58–59) | Mike Cohn & James Grenning

El **Planning Poker** es una técnica ágil, colaborativa y gamificada creada por James Grenning y popularizada por ****Mike Cohn****. Combina tres técnicas clásicas de estimación: **opinión de expertos**, **analogía (comparación relativa)** y **desagregación**.

Poker Planning

Prerrequisitos:

- Lista de features/stories a ser estimadas
- Cada estimador tiene un mazo de cartas.

Pasos:

- Determine la **base story** (la canónica) que será usada para comprar con las otras stories. Digamos, Story Z.
 - La story a ser estimada se lee a todo el equipo.
 - Los estimadores discuten la story, haciendo preguntas al product owner (las que se necesiten).
 - Cada estimador selecciona una carta y pone la carta boca abajo en la mesa.
 - Cuando todos pusieron las cartas, las mismas se exponen al mismo tiempo.
 - Si todos los estimadores selecciona el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos. Después de la charla, GOTO to 1.3.
- Se toma la próxima story, se discute con el product owner.



Mazo de cartas de Planning Poker basado en Fibonacci Modificada

¿QUIÉNES ESTIMAN?

ÚNICAMENTE los integrantes del Developers Team (las personas que efectivamente programarán y probarán el incremento). El Product Owner participa presentando la necesidad y respondiendo dudas de negocio, pero **no vota ni negocia los puntos**.

LA DINÁMICA PASO A PASO:

- Determinación de la historia canónica:** Se establece la historia de referencia (Story Z = 1 SP).
- Lectura de la Historia:** El Product Owner lee la User Story y detalla sus Criterios de Aceptación.
- Ronda de aclaración de dudas:** El equipo formula preguntas técnicas y funcionales al PO.
- Votación secreta (a ciegas):** Cada miembro selecciona una carta de su mazo y la coloca boca abajo sobre la mesa. Nadie puede emitir opiniones en voz alta para **eliminar el sesgo de anclaje y de autoridad** (evitar que la opinión del desarrollador más senior condicione a los demás).
- Revelación simultánea:** A la cuenta de tres, todos los integrantes muestran su carta al mismo tiempo.
- Debate enfocado en los extremos:**
 - Si la mayoría coincide, se adopta ese número por consenso.
 - Si hay divergencia notoria (ejemplo: votos de 2 y de 13), **SOLO argumentan los dos extremos:**
 - El voto más bajo (2):* Explica si descubrió una solución sencilla, una librería existente o un atajo técnico que los demás no vieron.
 - El voto más alto (13):* Explica qué riesgos ocultos, complejidades arquitectónicas o casos de borde identificó.
- Revotación y Consenso:** Tras un intercambio breve de 2 minutos, se vuelve a votar a ciegas. Habitualmente, en la segunda ronda se alcanza el consenso natural.

0	1/2	1	2
3	5	8	13
20	40	100	?

- 0:** Quizás ud. no tenga idea de su producto o funcionalidad en este punto.
- 1/2 - 1:** funcionalidad pequeña (usualmente cosmética).
- 2-3:** funcionalidad pequeña a mediana. Es lo que queremos. 😊
- 5:** Funcionalidad media. Es lo que queremos 😊
- 8:** Funcionalidad grande, de todas formas lo podemos hacer, pero hay que preguntarse sino se puede partir o dividir en algo más pequeño. No es lo mejor, pero todavía 😊
- 13:** Alguien puede explicar por que no lo podemos dividir?
- 20:**Cuál es la razón de negocio que justifica semejante story y más fuerte aún, por qué no se puede dividir?.
- 40:** no hay forma de hacer esto en un sprint.

Significado de los valores de Fibonacci en Planning Poker

DECODIFICACIÓN DE LOS VALORES EN PLANNING POKER (FILMINA 21 DE CÁTEDRA):

- 0** : Funcionalidad trivial que no requiere esfuerzo de desarrollo real.
- 0.5 - 1** : Funcionalidad sumamente pequeña o cambio cosmético.
- 2 - 3** : Funcionalidad pequeña a mediana. **Es el rango ideal para un Sprint** 😊.
- 5** : Funcionalidad media. Muy buen tamaño para trabajar con soltura 😊.
- 8** : Funcionalidad grande. Es viable de realizar, pero hay que preguntarse si no conviene dividirla.

- **13** : Límite superior tolerable en un Sprint. «¿Alguien puede explicar por qué no la podemos dividir?».
- **20** : Demasiado grande para un Sprint. Exige justificación extraordinaria de negocio; debe partirse en historias menores.
- **40** : Épica definitiva. Imposible completarla con calidad dentro de un Sprint.
- **100** : Confirmación de que algo está muy mal concebido o es inabordable. Mejor ni arrancar.
- **?** : Desconocimiento total o falta absoluta de información. **Candidata obligada a Spike.**
-  : Petición de pausa o descanso para el equipo.

LA MÉTRICA DE VELOCIDAD (VELOCITY):

- La velocidad **NO** se estima: se **CALCULA** empíricamente.
- Es la suma de los Story Points de aquellas Historias de Usuario que fueron **completadas al 100% (según la DoD)** y **formalmente aceptadas por el Product Owner** al finalizar la iteración.
- **Gestión binaria:** Las historias a medio terminar valen **0 puntos** de velocidad.
- **Propiedad auto-correctiva:** La velocidad empírica absorbe y corrige los errores de estimación del equipo con el transcurso de los Sprints. Si el equipo tiende a sobreestimar o subestimar, el promedio histórico estabiliza el ritmo y vuelve predecibles las proyecciones.

Ecuación Central de la Estimación Ágil

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 26–28) | Resumen Teórico Final (Pág. 58) | Apuntes de Clase

En los marcos de trabajo ágiles **nunca se estima el tiempo directamente**. Se estima la magnitud del trabajo (tamaño en SP), se mide empíricamente la velocidad real del equipo y se calcula matemáticamente el cronograma:

$$\text{Duración (en iteraciones)} = \frac{\text{Tamaño Total del Backlog (Story Points)}}{\text{Velocidad Promedio (Story Points por iteración)}}$$

- **Los Sprints son de duración fija (Timeboxed):** La fecha de finalización del Sprint jamás se posterga.
- Si la velocidad observada es menor a la calculada, **se negocia el alcance con el Product Owner**, pero jamás se recorta la calidad técnica ni se dilata la duración del ciclo.

Reflexión de Cátedra: ¿Por qué las Estimaciones Ágiles son Ágiles?

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diap. 29) | Resumen Teórico Final (Pág. 60) | Filosofía Lean

Cuatro fundamentos epistemológicos que distinguen a la estimación ágil del modelo predictivo:

1. **Son Relativas y no Absolutas (Tamaño vs. Horas):** Eliminan la falsa precisión de las horas lineales. Aprovechan la capacidad cognitiva humana de comparar magnitudes relativas frente a una historia canónica, incorporando complejidad, esfuerzo e incertidumbre.
2. **Son Just-in-Time y combaten el Cono de la Incertidumbre:** No se desperdicia tiempo estimando todo el Product Backlog al inicio. Se estima con detalle únicamente lo que entrará al próximo Sprint, dejando que la certeza emerja con el software funcionando y el feedback de cada incremento.
3. **Son Colectivas y Descentralizadas:** Estiman las personas que ejecutarán el trabajo (los desarrolladores). A través del Planning Poker se neutraliza el sesgo de anclaje, se visibilizan riesgos tempranos y se genera una visión técnica compartida.

4. **Se basan en el Empirismo (Separación de Estimación y Duración):** La estimación solo mide tamaño relativo; la duración es una consecuencia matemática gobernada por la velocidad histórica del equipo, la cual absorbe y autocorriges naturalmente los desvíos sprint tras sprint.

Diferencias entre Estimaciones Tradicionales vs. Ágiles en Cuatro Dimensiones

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 30–31) | Resumen Teórico Final (Págs. 60–61) | Cuadro Comparativo de Cátedra

A partir de las directivas impartidas por los profesores de la parte práctica y el marco teórico de la materia, la distinción entre estimaciones tradicionales y ágiles se estructura rigurosamente sobre cuatro dimensiones fundamentales:

1. DIMENSIÓN TAMAÑO (SIZE):

- **Enfoque Tradicional:** Qué tan grande es lo que se va a construir. En ambientes tradicionales se mide según la **cantidad de funcionalidades del software**, módulos descritos en la ERS, número de pantallas o métricas como Líneas de Código (SLOC) y Puntos de Función.
Observación Crítica de Cátedra: La primera estimación jamás puede realizarse mirando el código fuente porque este aún no existe. Mirar líneas de código tentativas puede servir como puntapié referencial, pero **no es una medida representativa ni significativa de la funcionalidad entregada al cliente**.

- **Enfoque Ágil:** El tamaño se estima sobre las User Stories utilizando una medida abstracta y relativa: los **Story Points (SP)**.

La estimación se formula matemáticamente como: **Medida = N × [UM]**, donde **[UM]** es la Unidad de Medida (Story Point) y **N** es el número asignado por consenso del equipo de desarrollo.

Psicología de la Comparación: Los seres humanos somos pésimos determinando magnitudes absolutas, pero somos **altamente competentes comparando**. Que una US sea más grande o más chica que otra refleja la combinación inseparable de tres factores:

- **Complejidad:** Qué tan difícil es el problema algorítmico o de dominio a resolver (complejidad del negocio o de las tecnologías).
- **Esfuerzo:** Horas-persona lineales requeridas para completar la historia de punta a punta.
- **Incertidumbre:** Grado de información que el equipo desconoce sobre las reglas de negocio o la tecnología a aplicar.

La Historia Canónica (Pivote): Para calibrar el metro patrón, el equipo define una historia representativa bien comprendida (ej. 2 o 3 SP) y dimensiona todas las demás por comparación directa contra ella.

Escala de Fibonacci y la Regla del 13: Se utiliza la serie de Fibonacci (1, 2, 3, 5, 8, 13, 20...) porque su crecimiento exponencial modela la incertidumbre creciente. **LA REGLA DE ORO DEL VALOR 13:** El valor 13 indica taxativamente que la Historia de Usuario **NO cumple con la 'S' (Small) del modelo INVEST** (comprometiendo además la Estimabilidad y Testeabilidad). Dicha historia **DEBE DIVIDIRSE OBLIGATORIAMENTE (Splitting)** en historias más pequeñas antes de ingresar a un Sprint.

2. DIMENSIÓN ESFUERZO (EFFORT):

- **Enfoque Tradicional:** Se mide en **Horas-Persona Lineales (HP)**. Representa el tiempo neto que una persona tardaría en realizar la actividad de forma secuencial. Comete la falacia lineal de asumir que si una tarea toma 100 horas, 100 desarrolladores la resolverán en 1 hora, ignorando dependencias técnicas y fricciones de comunicación (Ley de Brooks).
- **Enfoque Ágil:** El esfuerzo es solo un componente dentro de los Story Points a nivel macro. A nivel micro (Sprint Backlog), una vez comprometidas las historias, los desarrolladores desglosan las tareas técnicas y pueden estimarlas en **horas ideales** para monitorear el avance diario en el Sprint Burn-down Chart.

3. DIMENSIÓN TIEMPO Y CALENDARIO (SCHEDULE):

- **Enfoque Tradicional:** Se mide en días hábiles, semanas y meses calendario, mapeados en Diagramas de Gantt y cálculo del Camino Crítico (CPM). Considera feriados, licencias del personal y busca fijar una fecha inmutable de entrega final.
- **Enfoque Ágil:** El tiempo se organiza en iteraciones fijas innegociables (**Timeboxes de 2 a 4 semanas**). La fecha de entrega se proyecta empíricamente mediante la **Velocidad histórica del equipo** (promedio de Story Points terminados según la Definición de Hecho por Sprint) graficada en *Release Burn-down Charts*.

4. DIMENSIÓN COSTO (COST):

- **Enfoque Tradicional:** Presupuesto derivado de multiplicar las horas-persona estimadas de cada perfil por sus tarifas horarias, sumando costos fijos de infraestructura. Cualquier modificación en el alcance altera automáticamente el costo contractual, generando resistencia.
- **Enfoque Ágil:** El costo por Sprint es **prácticamente constante y predecible** (*Burn Rate* = salarios del equipo multidisciplinario estable más costos operativos). El presupuesto se optimiza flexibilizando el alcance: se entrega software funcionando en cada sprint, maximizando el retorno de inversión sin comprometer la calidad.

Dimensión	Estimación Tradicional (Predictiva)	Estimación Ágil (Empírica)
1. Tamaño (Size)	Absoluto y Funcional: Funcionalidades de la ERS, pantallas, módulos, SLOC o Puntos de Función.	Relativo y Multidimensional: Story Points (N × [UM]) basados en Complejidad, Esfuerzo e Incertidumbre. Fibonacci y US Canónica.
2. Esfuerzo (Effort)	Horas-Persona Lineales (HP): Tiempo secuencial de una persona. Riesgo de paralelismo irreal.	Compuesto en SP: Encapsulado a nivel macro; desglosado en horas ideales en tareas del Sprint Backlog.
3. Calendario (Time)	Cronograma Rígido: Diagramas de Gantt, Camino Crítico (CPM) y fecha de fin fija.	Timeboxes Fijos (Sprints): Duración constante (2-4 semanas). Proyección empírica mediante la Velocidad .
4. Costo (Cost)	Presupuesto Cerrado / Variable: Derivado de horas por perfiles. Renegociaciones contractuales.	Costo Fijo y Predecible (Burn Rate): Equipo estable; el alcance se adapta al presupuesto disponible.
Regla de Crisis	Si cambian tiempo o costos sin tocar alcance, se degrada la Calidad .	El tiempo y costo son fijos; si algo debe ajustarse, se flexibiliza el Alcance sin tocar la Calidad (DoD) .

Los Tres Niveles de Planificación en Ambientes Ágiles

Fuentes de Cátedra: Filmina Estimaciones Ágiles | Resumen Teórico Final (Págs. 56–57) | Mike Cohn (Agile Estimating and Planning)

La planificación en Agile no es un evento único al inicio, sino una disciplina continua guiada por la máxima: «*Planning is everything. Plans are nothing*» (Lo valioso es el acto dinámico de planificar y reflexionar, no el plan estático escrito en piedra). Coexisten tres niveles de planificación:

Nivel de Planificación	Horizonte Temporal	Objetivo y Dinámica	Unidad de Medida
1. Planificación de Release (Release Planning)	Mediano plazo (3 a 6 meses)	Se definen qué Temas, Épicas y User Stories formarán parte de la próxima entrega a producción. Se deriva la duración en Sprints mediante la fórmula: $\text{Cantidad de Sprints} = \frac{\text{Total Story Points}}{\text{Velocidad Promedio}}$.	Story Points
2. Planificación de Iteración (Sprint Planning)	Corto plazo (1 a 4 semanas)	El equipo toma los ítems de mayor prioridad del Product Backlog hasta saturar su capacidad estimada según su velocidad histórica. Descompone cada User Story en tareas técnicas concretas de ingeniería.	Story Points (US) Horas ideales (Tareas)
3. Planificación Diaria (Daily Scrum)	24 horas (diario)	Reunión de sincronización de 15 minutos de pie (*time-boxed*). El equipo coordina esfuerzos respondiendo: ¿Qué hice ayer? ¿Qué haré hoy? ¿Qué impedimentos tengo?	Compromiso diario

Estimaciones en la Filosofía Lean y en Proyectos Pequeños

Fuentes de Cátedra: Filmina Estimaciones Ágiles | Resumen Teórico Final (Págs. 59–60) | Apuntes de Clase

1. EL ENFOQUE LEAN FRENTE A LAS ESTIMACIONES (#NOESTIMATES):

- En Lean Software Development se adopta una postura más extrema que en Scrum clásico: **las estimaciones detalladas son consideradas potencialmente un Desperdicio (*Muda*)**.
- ¿Por qué? Porque tienen una probabilidad altísima de no cumplirse: en contextos de cambio continuo, cualquier variable que se altere invalida por completo el esfuerzo invertido en estimar.
- Lean prefiere enfocar su energía en **partir las historias a un tamaño uniformemente pequeño** y optimizar las métricas de flujo real del sistema: *Throughput* (cantidad de historias terminadas por unidad de tiempo) y *Lead Time* (tiempo desde que se pide hasta que se entrega).

2. PARTICULARIDADES DE LAS ESTIMACIONES EN PROYECTOS PEQUEÑOS:

- Inaplicabilidad de Datos de la Industria:** En proyectos pequeños, nunca deben utilizarse modelos paramétricos basados en métricas globales de la industria, ya que los promedios macro distorsionan la realidad.
- Uso Imperativo de Datos Históricos Propios:** Las estimaciones deben fundamentarse exclusivamente en registros históricos de proyectos similares realizados por la propia organización.
- Alta Dependencia de los Individuos:** En proyectos de escala reducida, la variabilidad del resultado depende de manera casi absoluta de las capacidades técnicas, experiencia y química del reducido grupo de desarrolladores asignado.

Criterios de Reestimación: ¿Cuándo Sí y Cuándo NO Reestimar?

Fuentes de Cátedra: Filmina Estimaciones Ágiles (Diaps. 32–34) | Resumen Teórico Final (Pág. 56) | Mike Cohn

Un aspecto técnico fundamental de examen evaluado en la página 56 del resumen histórico:

- **¿Cuándo SÍ se debe reestimar una User Story?**

Únicamente cuando **la percepción del tamaño relativo de la historia ha cambiado** porque el equipo descubrió información que la vuelve sustancialmente más grande o más chica respecto a la historia canónica. Se reestima para preservar la consistencia interna del metro patrón.

- **¿Cuándo NO se debe reestimar?**

NUNCA se debe reestimar una historia simplemente porque el equipo avanza más lento de lo previsto o porque no se está cumpliendo el cronograma.

Si el equipo rinde menos de lo esperado, no es que el tamaño del producto haya cambiado: lo que cambió es la **velocidad del equipo**. Es la métrica de velocidad la que debe registrar la caída para que el cálculo matemático de la duración se ajuste automáticamente.